

AFRICAN LEADERSHIP UNIVERSITY

Introduction to Blockchain Development

Extending the Blockchain-Based Library Book Lending Tracker

Token Rewards, Transaction Models and Mining Simulation

TECHNICAL REPORT

Student	Aman Abraha Kasa
Course	Introduction to Blockchain Development
Institution	African Leadership University
Assignment	Assignment 2 [F:A] – Extending the Library Book Lending Tracker
Date	October 2026
GitHub	github.com/Aman-Kasa/formativeblk/tree/main/formative-2
Demo video	https://youtu.be/8pMnrEKcySE

Table of Contents

1 Introduction	4
1.1 Background	4
1.2 Objectives	4
1.3 What changed from Formative 1	4
2 System Design	5
3 Updated Block Structure	6
4 How Lending Events Trigger Token Transactions	7
4.1 The reward rules	7
4.2 Late returns and overdue books	8
5 The Pending Pool and How Mining Confirms Blocks	9
6 The UTXO Model	12
6.1 Data structures	12
6.2 Rewards, fees and change outputs	12
7 The Account-Based Model	14
7.1 Accounts, debits and credits	14
7.2 Nonces	14
7.3 Transaction history	14
8 Comparing the Two Models	16
9 Mining Simulation Methodology	17
9.1 Proof of work and difficulty	17
9.2 Solo mining	18
9.3 Pool mining and contribution metrics	19
9.4 Cloud mining	20
10 Security and Immutability	23
10.1 How the chain is protected	23
10.2 Tamper demonstrations	23
10.3 Key management	24
10.4 Limitations	24
11 Testing and Edge-Case Results	25
11.1 Edge cases required by the brief	25
11.2 Other results	26
12 Design Choices and Assumptions	27
13 Challenges Encountered and Solutions	28
13.1 Balances that did not survive a restart	28
13.2 Duplicate transaction IDs	28
13.3 Fair pool contribution	28
13.4 Testing something random	28
13.5 Demonstrating late and overdue books	28
13.6 A key-handling bug carried over from Formative 1	28

14 Conclusion	3
Appendix A: Building and Running	3

1 Introduction

1.1 Background

Formative 1 built a command-line Library Lending Chain in C. Every borrow and return became a block that was hashed with SHA-256, linked to the block before it, signed with an ECDSA key, and checked against a book and member registry. As soon as a librarian typed a command, the block was added to the chain.

Formative 2 turns that record-keeping system into a small token economy. Returning a book now earns the member coins, the reward is a real transaction with its own SHA-256 identifier, and nothing reaches the chain until a miner confirms it with proof of work. This report explains how each part was built, shows the program running, and gives the test results for the edge cases the brief asks for.

1.2 Objectives

- Make borrowing and returning trigger blockchain transactions: 10 coins for an on-time return, 5 for a late return, and nothing for a book that stays overdue.
- Send every lending record to a pending pool and add it to the chain only after mining.
- Implement two ledger models, UTXO and account-based, that use the registry's member IDs as account identifiers, with fees, change outputs, nonces and transaction history.
- Implement proof of work with an adjustable difficulty (1–4) and show how difficulty changes the mining effort.
- Simulate solo, pool and cloud mining against the same pending pool, and report how the rewards are shared and whether a rental pays off.

1.3 What changed from Formative 1

Area	Formative 1	Formative 2
Adding a record	Appended to the chain immediately	Queued in a pending pool; appended only after mining
Block fields	index, timestamp, book, member, action, previous_hash, signature, hash	All of those, unchanged, plus <code>token_reward</code> , <code>tx_id</code> , <code>difficulty</code> and <code>nonce</code>
Hash	Any SHA-256 value	Must start with <i>d</i> zero hex characters (proof of work)
Actions	BORROWED, RETURNED	BORROWED, RETURNED and OVERDUE
Tokens	None	Rewards, member-to-member transfers, UTXO and account ledgers
Mining	None	Solo, pool and cloud mining with reward accounting
Source files	6 modules	12 modules (6 new: <code>mining</code> , <code>utxo</code> , <code>account</code> , <code>ledger</code> , <code>display</code> , <code>simclock</code>)

A note on wording. Parts of the brief still say "attendance" and "absent student", left over from an older version of the assignment. In this report they mean lending: "marking attendance" is a borrow or return, and "an absent student generates no transaction" is an overdue book that earns no reward.

2 System Design

Figure 1 shows the whole flow. A lending command is checked against the registries and the book's current loan state. If it is valid, the reward rule decides the coins and the record goes into the pending pool. A mining command takes records from the front of the pool, runs proof of work on each one, signs the winning hash and appends the block. Only then is the reward applied to the member ledger. Every time the program starts, the chain is validated again from the genesis block.

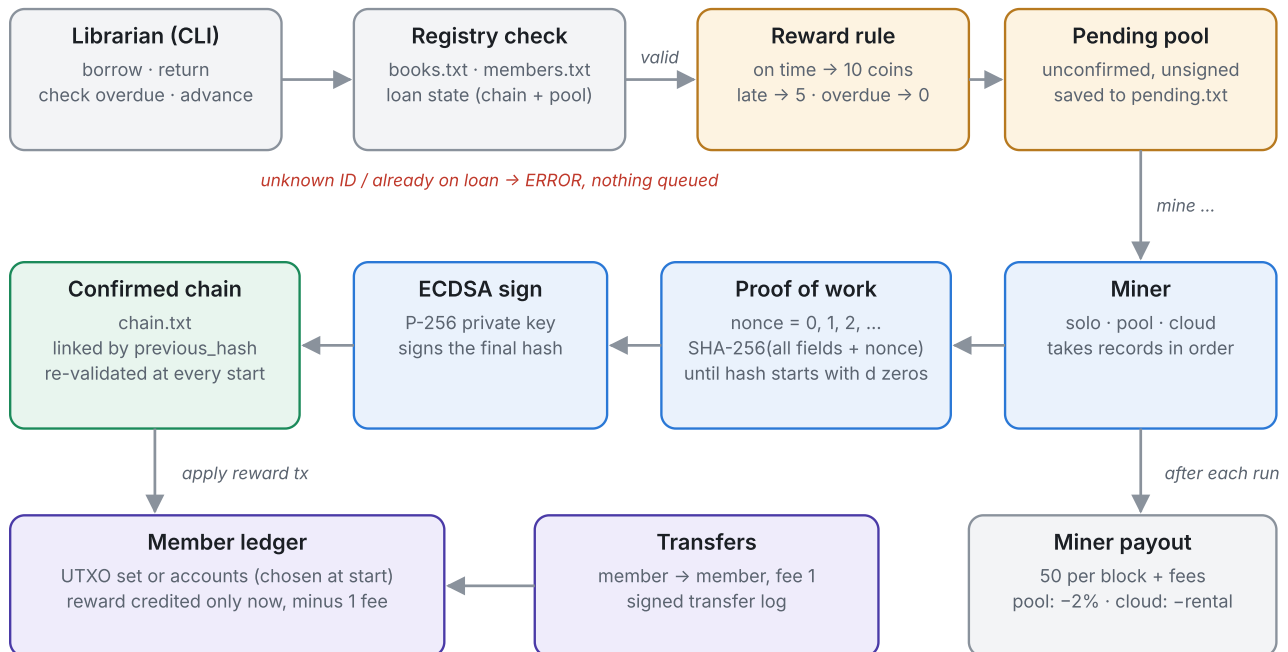


Figure 1. System design: from a lending command, through the pending pool and mining, to the chain and the member ledger

The code is split so that each file has one job:

Module	Responsibility
<code>main.c</code>	Arguments, model choice, loading and verifying the chain, pending pool and transfer log
<code>cli.c</code>	Command loop and argument checking
<code>blockchain.c</code>	Lending rules, pending pool, hashing, proof of work, validation
<code>mining.c</code>	Solo, pool and cloud mining, and reward accounting
<code>utxo.c</code> / <code>account.c</code>	The two ledger models
<code>ledger.c</code>	Model selection, signed transfer log, replay on startup
<code>persistence.c</code>	Chain and pending pool files
<code>crypto.c</code>	SHA-256, ECDSA key handling, signing and verifying (OpenSSL 3)
<code>registry.c</code>	books.txt and members.txt (unchanged from Formative 1)
<code>display.c</code> , <code>simclock.c</code> , <code>config.h</code>	Shared output, the simulated clock, and every tunable number

The confirmed chain, the pending pool, the UTXO set and each account's history are all singly linked lists of structs allocated with `calloc` and released on exit. The test suite runs the program under AddressSanitizer and LeakSanitizer to confirm there are no leaks or invalid memory accesses (Section 11).

3 Updated Block Structure

The brief requires two new fields and says every Formative 1 field must stay unchanged. Proof of work needs two more: a `nonce` the miner can change, and the `difficulty` the block was mined at. Difficulty is stored in each block because it can be changed between mining runs, so validation must check every block against the target it was actually mined at.

Field	Type	Description
<code>index</code> , <code>timestamp</code>	<code>int</code> , <code>time_t</code>	Position in the chain; time of the lending event
<code>book_id</code> , <code>book_title</code>	<code>char[20]</code> , <code>char[80]</code>	Validated against, and copied from, the book registry
<code>member_id</code> , <code>member_name</code>	<code>char[20]</code> , <code>char[50]</code>	Validated against, and copied from, the member registry
<code>action</code>	<code>char[10]</code>	GENESIS, BORROWED, RETURNED or OVERDUE
<code>previous_hash</code>	<code>char[65]</code>	Hash of the block before (64 zeros for genesis)
<code>signature</code>	unsigned <code>char[72]</code>	ECDSA signature of the block's hash (length kept in <code>sig_len</code>)
<code>hash</code>	<code>char[65]</code>	SHA-256 of all the hashed fields; must meet the difficulty
<code>token_reward</code>	<code>int</code>	New. 10 for an on-time return, 5 for a late return, 0 otherwise
<code>tx_id</code>	<code>char[65]</code>	New. SHA-256 of the reward transaction; empty when there is no reward
<code>difficulty</code>	<code>int</code>	New (proof of work). Leading zeros this block's hash must have
<code>nonce</code>	unsigned long	New (proof of work). The counter the miner increments

The block hash is SHA-256 over this string, built in a fixed order by `serialize_for_hash()` :

```
index|timestamp|book_id|book_title|member_id|member_name|action|previous_hash|difficulty|nonce|token_reward|tx_id
```

Because `token_reward` and `tx_id` are part of the hash input, a reward cannot be changed after mining without changing the hash, which then fails both the hash check and the proof-of-work check. Figure 2 shows three real blocks from the run in Figure 6.

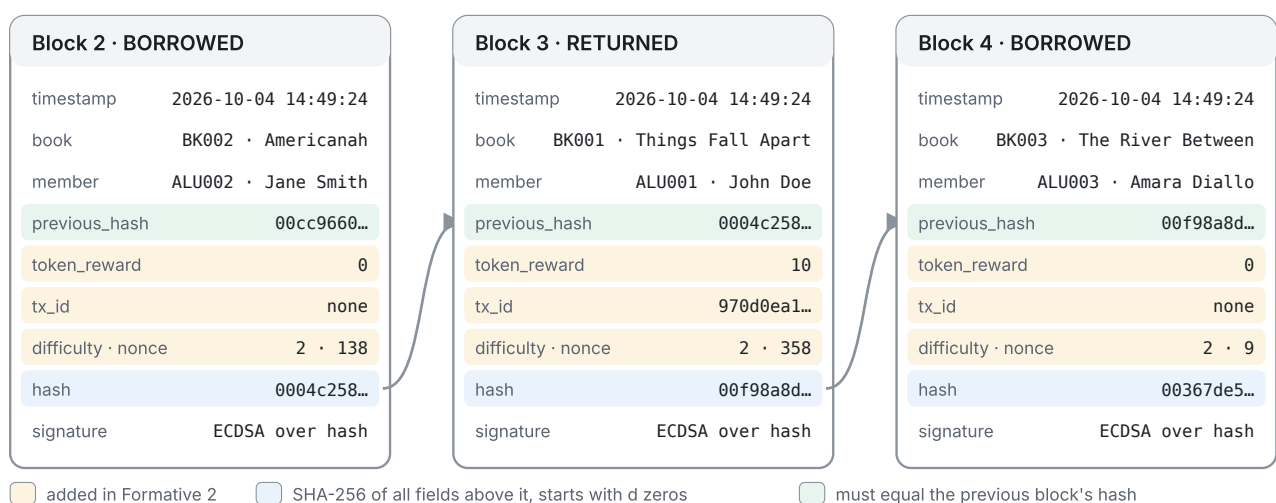


Figure 2. Block structure and chain linkage (values from a real run; hashes shortened)

4 How Lending Events Trigger Token Transactions

4.1 The reward rules

Event	Command	Action	token_reward	Transaction
Borrow	borrow BK001 ALU001	BORROWED	0	None
Return within 14 days	return BK001	RETURNED	10	SYSTEM → member, 10 – 1 fee
Return after 14 days	return BK001	RETURNED	5	SYSTEM → member, 5 – 1 fee
Still out after 14 days	check overdue	OVERDUE	0	None is created

When a book is returned, `lending_return()` finds the book's open loan (the latest BORROWED record, looking at the chain first and then the pending pool), compares the time since the borrow with the 14-day loan period, and sets the reward. It then builds the reward transaction and stores its identifier in the block:

```
tx_id = SHA-256("REWARD|SYSTEM|" + member_id + "|" + book_id + "|" + reward + "|" + fee + "|" + timestamp)
```

The transaction is created at return time but **applied only when its block is mined**. In Figure 3, John Doe's on-time return creates a 10-coin transaction with its `tx_id`, yet his balance is still 0 because the record is only pending. The same figure shows the registry checks from Formative 1 still working: an unknown book ID, a second borrow of a book already on loan, and a return of a book that was never borrowed are all rejected and nothing is queued.

```
~/formativeblk/formative-2

$ ./lending_tracker --model utxo --seed 7
Registries loaded: 3 book(s), 3 member(s).
Loaded chain: 1 block(s), integrity verified.
Transaction model: UTXO. Balances rebuilt from confirmed rewards and the signed transfer log.
Library Lending Chain (Formative 2) - UTXO model, difficulty 2. Type 'help' for commands.
> borrow BK001 ALU001
PENDING: BK001 borrowed by ALU001 - queued (not yet on the chain; 1 record(s) waiting to be mined).
> borrow BK002 ALU002
PENDING: BK002 borrowed by ALU002 - queued (not yet on the chain; 2 record(s) waiting to be mined).
> borrow BK999 ALU001
ERROR: Book or Member not found
> borrow BK001 ALU003
ERROR: this book is already on loan
> return BK003
ERROR: this book was never borrowed, or has already been returned
> return BK001
PENDING: BK001 returned by ALU001 - ON-TIME return, reward transaction of 10 coins created
tx_id=970d0ea103cbb13796b6acefad313c0a2a25f062d46d9803ee9abe2af899673c
The member is credited only after this record is mined.
> pending
Pending pool: 3 unconfirmed record(s) waiting to be mined
# Action Book Title Member Time Reward tx_id
1 BORROWED BK001 Things Fall Apart ALU001 2026-10-04 14:49:24 0 none
2 BORROWED BK002 Americanah ALU002 2026-10-04 14:49:24 0 none
3 RETURNED BK001 Things Fall Apart ALU001 2026-10-04 14:49:24 10 970d0ea103cbb137...
> balances
UTXO set (unspent outputs):
ID Owner Amount Created by (txid:vout)
(empty)
Balances: ALU001=0, ALU002=0, ALU003=0
> exit
```

Figure 3. Lending commands go to the pending pool. The on-time return creates a 10-coin reward transaction, but the balance stays 0 until mining.

4.2 Late returns and overdue books

Waiting two real weeks is not practical in a demonstration, so the program has a simulated clock. `advance <days>` moves it forward, and every lending event is stamped with the simulated time, so the on-time, late and overdue decisions are consistent with the timestamps stored in the blocks. In Figure 4, after 20 simulated days, Jane Smith's return of *Americanah* is late and earns 5 coins. Amara Diallo still has *The River Between*, so `check overdue` queues an OVERDUE record with `token_reward = 0` and no `tx_id`: no transaction exists for it. An overdue book still counts as on loan, so it cannot be borrowed by someone else, and it can still be returned later as a late return.

```
~ /formativeblk/formative-2

$ ./lending_tracker --model utxo --seed 7
Registries loaded: 3 book(s), 3 member(s).
Loaded chain: 4 block(s), integrity verified.
Transaction model: UTX0. Balances rebuilt from confirmed rewards and the signed transfer log.
Library Lending Chain (Formative 2) - UTX0 model, difficulty 2. Type 'help' for commands.
> borrow BK003 ALU003
PENDING: BK003 borrowed by ALU003 - queued (not yet on the chain; 1 record(s) waiting to be mined).
> advance 20
Simulated clock moved forward 20 day(s); now 20 day(s) ahead of real time.
> return BK002
PENDING: BK002 returned by ALU002 - LATE return, reward transaction of 5 coins created
tx_id=88b8f09b470bb4f10d0f34265d10fbaa2006f0cdfea4345cf487dee0992fc7e
The member is credited only after this record is mined.
> check overdue
PENDING: 1 OVERDUE record(s) queued. Overdue books earn no reward: token_reward = 0 and no transaction is created.
> pending
Pending pool: 3 unconfirmed record(s) waiting to be mined
# Action Book Title Member Time Reward tx_id
1 BORROWED BK003 The River Between ALU003 2026-10-04 14:49:24 0 none
2 RETURNED BK002 Americanah ALU002 2026-10-24 14:49:24 5 88b8f09b470bb4f1...
3 OVERDUE BK003 The River Between ALU003 2026-10-24 14:49:24 0 none
> exit
```

Figure 4. A late return earns 5 coins; an overdue book is queued as OVERDUE with no reward transaction.

5 The Pending Pool and How Mining Confirms Blocks

The pending pool is a linked list of unconfirmed records, saved to `data/pending.txt` after every change so queued work survives a restart. Records in the pool have no index, no previous hash and no signature yet, because those depend on where the block ends up in the chain. Every mining method confirms records the same way, in the order they were queued:

1. **Prepare.** `blockchain_prepare()` sets the block's `index` to the chain length, copies the last block's hash into `previous_hash`, records the current `difficulty`, and sets `nonce` to 0.
2. **Proof of work.** `blockchain_pow()` computes the SHA-256 hash, and if it does not start with d zeros, increments the nonce and tries again, counting every attempt.
3. **Sign.** The winning hash is signed with the library's ECDSA (P-256) private key. Because the hash covers every field, the signature does too.
4. **Append and apply.** The block is appended to the chain, its reward transaction (if any) is applied to the member ledger, and the UTXO set is printed in the UTXO model.
5. **Save.** Both `chain.txt` and the now smaller `pending.txt` are written, so a crash never loses or duplicates a record.

Figure 5 mines the three records queued in Figure 3. Each block reports the number of hash attempts it needed, and John Doe's reward is credited only when block 3 is confirmed: 10 coins minus the 1-coin fee gives him a 9-coin UTXO.

```

~/formativeblk/formative-2

$ ./lending_tracker --model utxo --seed 7
Registries loaded: 3 book(s), 3 member(s).
Loaded chain: 1 block(s), integrity verified.
Pending pool: 3 unconfirmed record(s) restored.
Transaction model: UTXO. Balances rebuilt from confirmed rewards and the signed transfer log.
Library Lending Chain (Formative 2) - UTXO model, difficulty 2. Type 'help' for commands.
> mine solo
=== Solo mining: 3 pending record(s), difficulty 2 (hash must start with 2 zeros) ===

Block 1: valid hash found after 394 attempts (nonce 393, 0.4 ms)
Confirmed:
[1] BORROWED "Things Fall Apart" (BK001) - John Doe (ALU001)
    time=2026-10-04 14:49:24 token_reward=0 tx_id=none (no reward transaction)
    hash=00cc9660020f19bf363b5c52dc7411800765c3aad58eb8c63e79fa65268a223d
    prev=0083c88e3a5a03da7962da41c4071a58ec3496882ede24c3d411f82b040d2e80
    difficulty=2 nonce=393 signature=VALID
UTXO set (unspent outputs):
  ID      Owner      Amount      Created by (txid:vout)
  (empty)
Balances: ALU001=0, ALU002=0, ALU003=0

Block 2: valid hash found after 139 attempts (nonce 138, 0.1 ms)
Confirmed:
[2] BORROWED "Americanah" (BK002) - Jane Smith (ALU002)
    time=2026-10-04 14:49:24 token_reward=0 tx_id=none (no reward transaction)
    hash=0004c2589ef2197cd954628c3e7f69f217a0354ad0c0b063229217bf172d2b90
    prev=00cc9660020f19bf363b5c52dc7411800765c3aad58eb8c63e79fa65268a223d
    difficulty=2 nonce=138 signature=VALID
UTXO set (unspent outputs):
  ID      Owner      Amount      Created by (txid:vout)
  (empty)
Balances: ALU001=0, ALU002=0, ALU003=0

Block 3: valid hash found after 359 attempts (nonce 358, 0.4 ms)
Confirmed:
[3] RETURNED "Things Fall Apart" (BK001) - John Doe (ALU001)
    time=2026-10-04 14:49:24 token_reward=10 tx_id=970d0ea103cbb13796b6acefad313c0a2a25f062d46d9803ee9abe2af899673c
    hash=00f98a8d85c865764da5d72c40026731b7b665e7ee55e7673f0aed233e5e6a9e
    prev=0004c2589ef2197cd954628c3e7f69f217a0354ad0c0b063229217bf172d2b90
    difficulty=2 nonce=358 signature=VALID
    -> reward transaction confirmed: ALU001 credited 10 - fee 1 = 9 coins
UTXO set (unspent outputs):
  ID      Owner      Amount      Created by (txid:vout)
  #1      ALU001      9          970d0ea103cbb137...:0
Balances: ALU001=9, ALU002=0, ALU003=0

--- Solo mining summary ---
Blocks confirmed      : 3
Total hash attempts   : 892 (avg 297 per block, expected ~256 at difficulty 2)
Time                  : 2.6 ms
Miner reward          : 3 x 50 block reward + 1 fees = 151.00 coins (all to the solo miner)
> exit

```

Figure 5. Solo mining confirms each pending record: hash attempts per block, the confirmed record, and the UTXO set after each block.

Figure 6 shows the confirmed records afterwards. Each one displays the book title, member name, action, timestamp, token reward and signature validity, as the brief requires, together with the hash, previous hash, difficulty and nonce. `validate chain` then re-checks every block.

```

~/formativeblk/formative-2

$ ./lending_tracker --model utxo --seed 7
Registries loaded: 3 book(s), 3 member(s).
Loaded chain: 7 block(s), integrity verified.
Transaction model: UTXO. Balances rebuilt from confirmed rewards and the signed transfer log.
Library Lending Chain (Formative 2) - UTXO model, difficulty 2. Type 'help' for commands.
> view records
Confirmed lending chain (7 blocks):
[0] GENESIS    time=2026-10-04 14:49:24
    hash=0083c88e3a5a03da7962da41c4071a58ec3496882ede24c3d411f82b040d2e80
    prev=0000000000000000000000000000000000000000000000000000000000000000
    difficulty=2  nonce=581  signature=VALID
[1] BORROWED   "Things Fall Apart" (BK001) - John Doe (ALU001)
    time=2026-10-04 14:49:24  token_reward=0  tx_id=none (no reward transaction)
    hash=00cc9660020f19bf363b5c52dc7411800765c3aad58eb8c63e79fa65268a223d
    prev=0083c88e3a5a03da7962da41c4071a58ec3496882ede24c3d411f82b040d2e80
    difficulty=2  nonce=393  signature=VALID
[2] BORROWED   "Americanah" (BK002) - Jane Smith (ALU002)
    time=2026-10-04 14:49:24  token_reward=0  tx_id=none (no reward transaction)
    hash=0004c2589ef2197cd954628c3e7f69f217a0354ad0c0b063229217bf172d2b90
    prev=00cc9660020f19bf363b5c52dc7411800765c3aad58eb8c63e79fa65268a223d
    difficulty=2  nonce=138  signature=VALID
[3] RETURNED   "Things Fall Apart" (BK001) - John Doe (ALU001)
    time=2026-10-04 14:49:24  token_reward=10  tx_id=970d0ea103cbb13796b6acefad313c0a2a25f062d46d9803ee9abe2af899673c
    hash=00f98a8d85c865764da5d72c40026731b7b665e7ee55e7673f0aed233e5e6a9e
    prev=0004c2589ef2197cd954628c3e7f69f217a0354ad0c0b063229217bf172d2b90
    difficulty=2  nonce=358  signature=VALID
[4] BORROWED   "The River Between" (BK003) - Amara Diallo (ALU003)
    time=2026-10-04 14:49:24  token_reward=0  tx_id=none (no reward transaction)
    hash=00367de532f10efbaf6ba7509f07a0b1083b32f7951e001229b25437a41dc892
    prev=00f98a8d85c865764da5d72c40026731b7b665e7ee55e7673f0aed233e5e6a9e
    difficulty=2  nonce=9  signature=VALID
[5] RETURNED   "Americanah" (BK002) - Jane Smith (ALU002)
    time=2026-10-24 14:49:24  token_reward=5  tx_id=88b8f09b470bb4f10d0f34265d10fbeaa2006f0cdfea4345cf487dee0992fc7e
    hash=00f3bfcfe8479776a3a50cca5faff66571e2dc7ec46d0ab5026539f9b5767824
    prev=00367de532f10efbaf6ba7509f07a0b1083b32f7951e001229b25437a41dc892
    difficulty=2  nonce=379  signature=VALID
[6] OVERDUE    "The River Between" (BK003) - Amara Diallo (ALU003)
    time=2026-10-24 14:49:24  token_reward=0  tx_id=none (no reward transaction)
    hash=00974db6afb577045fdee5eb6e63e2d0c696eb4fe0cdd7d28c446d32316695d
    prev=00f3bfcfe8479776a3a50cca5faff66571e2dc7ec46d0ab5026539f9b5767824
    difficulty=2  nonce=45  signature=VALID
> validate chain
Chain status: 7 block(s), VALID
every hash recomputes, every link matches, every block meets its proof-of-work
target, every tx_id matches its reward, and every signature verifies.
> exit

```

Figure 6. All confirmed records after mining, and a successful chain validation.

Validation (`blockchain_validate()`) walks the chain from the genesis block and stops at the first problem. For every block it checks, in order: the index sequence, that `previous_hash` matches the prior block's hash, that the recomputed hash matches the stored hash, that the hash meets the block's difficulty, that the stored `tx_id` matches the reward transaction recomputed from the block's fields, and that the ECDSA signature verifies with the public key.

6 The UTXO Model

6.1 Data structures

In the UTXO model, a member's coins are not a number stored anywhere. They are the set of unspent transaction outputs that name the member as owner. Each output is a node in a linked list:

```
typedef struct Utxo {
    int id; /* short id shown in the CLI: #1, #2, ... */
    char txid[65]; /* transaction that created this output */
    int vout; /* position within that transaction */
    char owner[MEMBER_ID_LEN]; /* a member ID from members.txt */
    long amount;
    int spent; /* set once the output is used as an input */
    struct Utxo *next;
} Utxo;
```

A member's balance is the sum of the unspent outputs they own (`utxo_balance()`). Every member starts with no outputs, so with a balance of 0.

6.2 Rewards, fees and change outputs

- **Reward.** A confirmed reward is a transaction with no inputs, like a coinbase transaction. It creates one output of `reward - 1` for the member, identified by the block's `tx_id`. The 1-coin fee goes to the miner.
- **Transfer.** `transfer <from> <to> <amount> [#id ...]` spends either the listed outputs or, if none are listed, the sender's oldest outputs until they cover `amount + 1`. The recipient gets an output for `amount`. If the inputs are worth more than `amount + fee`, the excess comes back to the sender as a **change output**. The inputs are then marked spent.
- **Validation.** All checks happen before anything changes, so a rejected transaction leaves the UTXO set untouched. A transfer is rejected if an input does not exist, is already spent (a **double spend**), belongs to someone else, or is listed twice, or if the inputs cannot cover `amount + fee`.

In Figure 7, John Doe's 9-coin output #1 pays 5 coins to Jane Smith. The fee is 1 coin, so 3 coins come back to him as change output #4. Trying to spend #1 again is rejected as a double spend, a 50-coin transfer is rejected because his inputs total only 3, and Amara Diallo cannot spend Jane Smith's output #2.

```
~/formativeblk/formative-2

$ ./lending_tracker --model utxo --seed 7
Registries loaded: 3 book(s), 3 member(s).
Loaded chain: 7 block(s), integrity verified.
Transaction model: UTXO. Balances rebuilt from confirmed rewards and the signed transfer log.
Library Lending Chain (Formative 2) - UTXO model, difficulty 2. Type 'help' for commands.
> utxos
UTXO set (unspent outputs):
  ID   Owner      Amount   Created by (txid:vout)
  #1   ALU001      9        970d0ea103cbb137...:0
  #2   ALU002      4        88b8f09b470bb4f1...:0
Balances: ALU001=9, ALU002=4, ALU003=0
> transfer ALU001 ALU002 5 #1
Transfer 03974568216a...: spent 1 input(s) worth 9 -> 5 to ALU002, 3 change back to ALU001, fee 1.
UTXO set (unspent outputs):
  ID   Owner      Amount   Created by (txid:vout)
  #2   ALU002      4        88b8f09b470bb4f1...:0
  #3   ALU002      5        03974568216aab30...:0
  #4   ALU001      3        03974568216aab30...:1
Balances: ALU001=3, ALU002=9, ALU003=0
> transfer ALU001 ALU003 1 #1
ERROR: double spend rejected - input #1 is already spent
> transfer ALU001 ALU002 50
ERROR: insufficient funds - inputs total 3, but amount 50 + fee 1 = 51
> transfer ALU003 ALU001 1 #2
ERROR: input #2 belongs to ALU002, not ALU003
> exit
```

Figure 7. UTXO transfer with a change output, followed by a rejected double spend, insufficient inputs, and spending someone else's output.

The full UTXO set, with every unspent output and its owner, is printed after each confirmed block (Figure 5) and on demand with `utxos` or `balances` .

7 The Account-Based Model

7.1 Accounts, debits and credits

In the account model, each registered member is an account that holds a balance directly:

```
typedef struct {
    char member_id[MEMBER_ID_LEN];
    long balance;
    long nonce;           /* next nonce this account must use */
    TxRecord *history_head; /* linked list of transactions */
    TxRecord *history_tail;
} Account;
```

A confirmed reward credits `reward - 1` to the member. A transfer debits `amount + 1` from the sender and credits `amount` to the recipient. A transfer is rejected if the sender's balance cannot cover `amount + fee`.

7.2 Nonces

Each account's nonce counts its outgoing transactions. A transfer must quote the sender's current nonce exactly, and the nonce increments after every successful transfer. A lower nonce is a reused nonce and is rejected as a replay. A higher nonce would skip transactions and is also rejected. This stops someone from submitting the same signed payment twice, which is the account model's protection against double spending.

7.3 Transaction history

Every account keeps a history log as a linked list of `TxRecord` nodes in memory, recording the kind, sender, recipient, amount, fee and nonce. A transfer is logged in both the sender's and the recipient's history. Rewards are logged with `SYSTEM` as the sender and no nonce. `history <member_id>` prints the full history for any member.

Figure 8 starts with John Doe holding 18 coins from two on-time returns. A transfer with nonce 0 succeeds. Reusing nonce 0 is rejected as a replay, nonce 3 is rejected for skipping ahead, and a 100-coin transfer is rejected for insufficient balance. Nonce 1 then succeeds, and both members' histories show the transactions.

```

~/formativeblk/formative-2

$ ./lending_tracker --model account --seed 7
Registries loaded: 3 book(s), 3 member(s).
Loaded chain: 5 block(s), integrity verified.
Transaction model: Account-based. Balances rebuilt from confirmed rewards and the signed transfer log.
Library Lending Chain (Formative 2) - Account-based model, difficulty 2. Type 'help' for commands.
> balances
Account balances:
  Member      Balance  Next nonce
  ALU001       18         0
  ALU002        0         0
  ALU003        0         0
> transfer ALU001 ALU002 5 0
Transfer: ALU001 -> ALU002, amount 5, fee 1, nonce 0. ALU001 now has 12 (next nonce 1).
> transfer ALU001 ALU002 1 0
ERROR: invalid nonce 0 for ALU001 - expected 1 (nonce already used: replay rejected)
> transfer ALU001 ALU002 1 3
ERROR: invalid nonce 3 for ALU001 - expected 1 (nonce skips ahead)
> transfer ALU001 ALU002 100 1
ERROR: insufficient balance - ALU001 has 12, needs amount 100 + fee 1 = 101
> transfer ALU001 ALU002 2 1
Transfer: ALU001 -> ALU002, amount 2, fee 1, nonce 1. ALU001 now has 9 (next nonce 2).
> history ALU001
Transaction history for ALU001 (balance 9, next nonce 2):
  Kind      Sender      Recipient  Amount  Fee  Nonce
  REWARD    SYSTEM      ALU001     9        1    -
  REWARD    SYSTEM      ALU001     9        1    -
  TRANSFER  ALU001      ALU002     5        1    0
  TRANSFER  ALU001      ALU002     2        1    1
> history ALU002
Transaction history for ALU002 (balance 7, next nonce 0):
  Kind      Sender      Recipient  Amount  Fee  Nonce
  TRANSFER  ALU001      ALU002     5        1    0
  TRANSFER  ALU001      ALU002     2        1    1
> balances
Account balances:
  Member      Balance  Next nonce
  ALU001        9         2
  ALU002        7         0
  ALU003        0         0
> exit

```

Figure 8. Account model: a valid transfer, a reused nonce, a skipped nonce, insufficient balance, and both members' transaction histories.

8 Comparing the Two Models

Aspect	UTXO model	Account model
What is stored	Individual coins (outputs) with an owner	One balance per member
Balance	Calculated: sum of unspent outputs	Read directly
Spending	Consumes whole outputs; excess returns as change	Subtracts the amount from the balance
Double-spend protection	An output can be spent only once (<code>spent</code> flag)	Each transfer must use the next nonce
Replay protection	Spent inputs cannot be reused	Reused or out-of-order nonces are rejected
History	Implicit: follow outputs back through txids	Explicit linked list per account
Privacy	Better: a member's coins are spread across outputs	Weaker: one balance per identity
Parallel processing	Easier: transactions with different inputs are independent	Harder: transfers from one account must be ordered by nonce
Simplicity	More bookkeeping (selection, change)	Simpler, closer to a bank account
Real-world example	Bitcoin	Ethereum

For a library, the account model is easier to explain to members and to audit by person: "John Doe has 9 coins". The UTXO model makes every coin's origin traceable back to the specific returned book that created it, and makes double spending structurally impossible. Both use the same member IDs from `members.txt`, start every member at 0, and are rebuilt from the chain at startup, so the same chain can be opened with either model.

9 Mining Simulation Methodology

9.1 Proof of work and difficulty

A block is valid only if its SHA-256 hash, written in hex, starts with d zeros, where d is the difficulty (1 to 4, default 2). Each hex character has 16 possible values, so a random hash meets the target with probability $1/16^d$, and the expected number of attempts is 16^d : 16, 256, 4,096 and 65,536. Difficulty is set at startup with `--difficulty` or during a session with the `difficulty` command, and values outside 1–4 are rejected.

To show how difficulty affects mining effort, 60 blocks were mined at each difficulty with the real program. Figure 9 shows the results: each step up multiplies the work by about 16, and the measured averages are close to the theoretical values. Individual blocks vary a lot, because finding a valid hash is random. At difficulty 4, one block needed only 579 attempts and another needed 369,431.

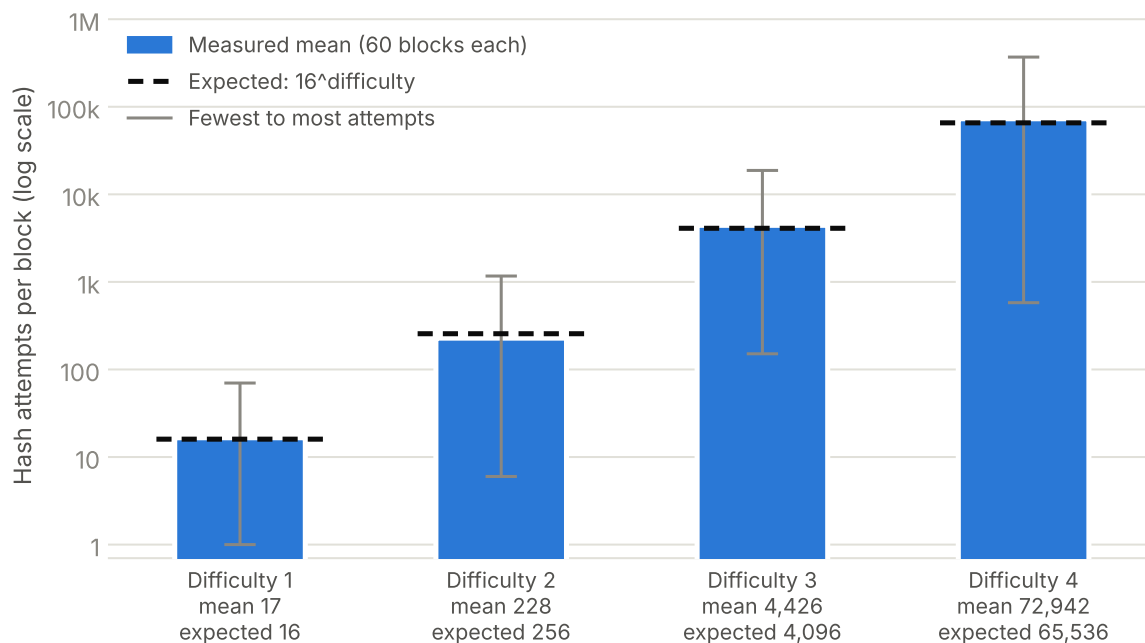


Figure 9. Hash attempts per block at each difficulty (60 blocks each, log scale)

Difficulty	Expected attempts	Measured mean	Median	Fewest – most	Mean time per block
1	16	17	11	1 – 70	0.03 ms
2	256	228	120	6 – 1,167	0.29 ms
3	4,096	4,426	2,742	151 – 18,750	4.42 ms
4	65,536	72,942	49,288	579 – 369,431	77.75 ms

Figure 10 shows the same effect in a single session: at difficulty 1 a block was found after 16 attempts, and at difficulty 4 the next block needed 69,595. An invalid difficulty of 7 is rejected.

```

~/formativeblk/formative-2

$ ./lending_tracker --model utxo --seed 7
Registries loaded: 3 book(s), 3 member(s).
Loaded chain: 1 block(s), integrity verified.
Transaction model: UTXO. Balances rebuilt from confirmed rewards and the signed transfer log.
Library Lending Chain (Formative 2) - UTXO model, difficulty 2. Type 'help' for commands.
> difficulty 1
Difficulty set to 1: blocks mined from now on need 1 leading zero (~16 attempts expected).
> borrow BK001 ALU001
PENDING: BK001 borrowed by ALU001 - queued (not yet on the chain; 1 record(s) waiting to be mined).
> mine solo
=== Solo mining: 1 pending record(s), difficulty 1 (hash must start with 1 zero) ===

Block 1: valid hash found after 16 attempts (nonce 15, 0.0 ms)
Confirmed:
[1] BORROWED "Things Fall Apart" (BK001) - John Doe (ALU001)
    time=2026-10-04 14:53:05 token_reward=0 tx_id=None (no reward transaction)
    hash=00a87c2e57a783c6a6161fa153bc502c61a068303fe34e2d8ecfc17398364f42
    prev=009544fdf32aaa034f41b0dbd7746afca5ce92fcc512fb8a6c7686dea62edbbd
    difficulty=1 nonce=15 signature=VALID
UTXO set (unspent outputs):
  ID   Owner      Amount   Created by (txid:vout)
  (empty)
Balances: ALU001=0, ALU002=0, ALU003=0

--- Solo mining summary ---
Blocks confirmed      : 1
Total hash attempts   : 16 (avg 16 per block, expected ~16 at difficulty 1)
Time                  : 1.3 ms
Miner reward          : 1 x 50 block reward + 0 fees = 50.00 coins (all to the solo miner)
> difficulty 4
Difficulty set to 4: blocks mined from now on need 4 leading zeros (~65536 attempts expected).
> return BK001
PENDING: BK001 returned by ALU001 - ON-TIME return, reward transaction of 10 coins created
    tx_id=601abd3813ccff0c28a05c81e4dacb86f3e80a44a1d267984418d6156437ede9
    The member is credited only after this record is mined.
> mine solo
=== Solo mining: 1 pending record(s), difficulty 4 (hash must start with 4 zeros) ===

Block 2: valid hash found after 69595 attempts (nonce 69594, 77.7 ms)
Confirmed:
[2] RETURNED "Things Fall Apart" (BK001) - John Doe (ALU001)
    time=2026-10-04 14:53:05 token_reward=10 tx_id=601abd3813ccff0c28a05c81e4dacb86f3e80a44a1d267984418d6156437ede9
    hash=000066952af9506142d04d5d9c548950c3e2ff2a40c008f334f2df0c4198f9ed
    prev=00a87c2e57a783c6a6161fa153bc502c61a068303fe34e2d8ecfc17398364f42
    difficulty=4 nonce=69594 signature=VALID
    -> reward transaction confirmed: ALU001 credited 10 - fee 1 = 9 coins
UTXO set (unspent outputs):
  ID   Owner      Amount   Created by (txid:vout)
  #1   ALU001      9        601abd3813ccff0c...:0
Balances: ALU001=9, ALU002=0, ALU003=0

--- Solo mining summary ---
Blocks confirmed      : 1
Total hash attempts   : 69595 (avg 69595 per block, expected ~65536 at difficulty 4)
Time                  : 78.2 ms
Miner reward          : 1 x 50 block reward + 1 fees = 51.00 coins (all to the solo miner)
> difficulty 7
ERROR: difficulty must be a whole number from 1 to 4.
> exit

```

Figure 10. Mining the same kind of block at difficulty 1 and difficulty 4.

9.2 Solo mining

One miner confirms every pending record and receives the full reward: 50 coins per block plus every transaction fee collected. The program prints the hash attempts and nonce for each block and a summary with total attempts, the expected attempts, time taken, and the miner's reward (Figure 5). Miner earnings are reported but are kept separate from the member ledgers, because miners are not library members.

9.3 Pool mining and contribution metrics

Pool mining (`mine pool [2-8]` , default 4 miners) works through the same pending pool. Each miner is given a random hash rate of 50 to 400 attempts per round. All miners hash at the same time. In each round, miner i searches its own slice of the nonce space, starting where miner $i-1$'s slice ends, so no two miners repeat each other's work.

If a miner finds a valid hash after k of its r attempts, it found it at time k/r through the round, and the miner with the earliest time wins the block. Each miner is credited only with the attempts it had made by that moment, so the contribution metric is fair: a fast miner is not penalised for where it sits in the list, and a slow miner is not credited for work it had not yet done.

Rewards are shared by contribution:

```
total reward      = blocks × 50 + transaction fees
pool fee          = 2% of total reward
distributable     = total reward - pool fee
miner's share     = miner_attempts / total_attempts
miner's reward    = share × distributable
```

In Figure 11, three blocks earn 151 coins. The 2% pool fee is 3.02 coins, leaving 147.98 to distribute. M4 had the highest hash rate and made 113 of the 310 attempts, so it receives 36.45% (53.94 coins), even though M1 found two of the three blocks. The table shows each miner's ID, hash rate, attempts, blocks found, share percentage and reward.

```

~/formativeblk/formative-2

$ ./lending_tracker --model utxo --seed 7
Registries loaded: 3 book(s), 3 member(s).
Loaded chain: 4 block(s), integrity verified.
Pending pool: 3 unconfirmed record(s) restored.
Transaction model: UTXO. Balances rebuilt from confirmed rewards and the signed transfer log.
Library Lending Chain (Formative 2) - UTXO model, difficulty 2. Type 'help' for commands.
> mine pool 4
=== Pool mining: 4 miners, 3 pending record(s), difficulty 2 ===
Miner M1 hash rate: 257 attempts/round
Miner M2 hash rate: 101 attempts/round
Miner M3 hash rate: 316 attempts/round
Miner M4 hash rate: 387 attempts/round

Block 4: found by M1 in round 1, 4% of the way through it (nonce 9)
Confirmed:
[4] BORROWED "The River Between" (BK003) - Amara Diallo (ALU003)
    time=2026-10-04 14:49:24 token_reward=0 tx_id=none (no reward transaction)
    hash=00367de532f10efbaf6ba7509f07a0b1083b32f7951e001229b25437a41dc892
    prev=00f98a8d85c865764da5d72c40026731b7b665e7ee55e7673f0aed233e5e6a9e
    difficulty=2 nonce=9 signature=VALID
UTXO set (unspent outputs):
ID   Owner      Amount   Created by (txid:vout)
#1   ALU001      9        970d0ea103cbb137...:0
Balances: ALU001=9, ALU002=0, ALU003=0

[ ... 26 lines omitted here: blocks 5 (late return) and 6 (overdue) confirmed the same way ... ]

--- Pool reward distribution ---
Total reward: 3 x 50 block reward + 1 fees = 151.00 coins
Pool fee (2%): 3.02 coins -> distributed to miners: 147.98 coins
Share = miner_attempts / total_attempts; reward = share x distributable reward

+-----+-----+-----+-----+-----+-----+
| Miner ID | Hash rate | Attempts | Blocks found | Share % | Reward |
+-----+-----+-----+-----+-----+-----+
| M1       | 257       | 74       | 2           | 23.87% | 35.32 |
| M2       | 101       | 31       | 0           | 10.00% | 14.80 |
| M3       | 316       | 92       | 1           | 29.68% | 43.92 |
| M4       | 387       | 113      | 0           | 36.45% | 53.94 |
+-----+-----+-----+-----+-----+-----+
| TOTAL    |           | 310      | 3           | 100.00% | 147.98 |
+-----+-----+-----+-----+-----+-----+

> balances
UTXO set (unspent outputs):
ID   Owner      Amount   Created by (txid:vout)
#1   ALU001      9        970d0ea103cbb137...:0
#2   ALU002      4        88b8f09b470bb4f1...:0
Balances: ALU001=9, ALU002=4, ALU003=0
> exit

```

Figure 11. Pool mining with four miners and the reward-sharing table after the 2% pool fee.

9.4 Cloud mining

In cloud mining (mine cloud <1-5>), the user rents a rig for a chosen number of rounds instead of mining directly. The rig makes 2,000 attempts per round. Proof of work on a block carries over between rounds, so a hard block can be finished in a later round. Each round:

- a fixed rental fee of 20 coins is charged;
- each block confirmed earns 50 coins plus its fees, minus a 10% maintenance fee;
- the program prints the round's attempts, blocks, reward, rental and maintenance, and the running totals;
- if the cumulative fees exceed the cumulative rewards, it prints a warning that the rental is unprofitable.

At the end it prints gross earnings, total fees paid and net profit. Figure 12 shows a profitable rental: two blocks earn 101 coins against 50.10 coins of fees, for a net profit of 50.90. Figure 13 shows an unprofitable one: at difficulty 4 a block needs about 65,536 attempts on average, but two rounds give only 4,000, so nothing is confirmed, 40 coins of rental are lost, and every round prints the warning.

```

~/formativeblk/formative-2

$ ./lending_tracker --model utxo --seed 7
Registries loaded: 3 book(s), 3 member(s).
Loaded chain: 1 block(s), integrity verified.
Pending pool: 2 unconfirmed record(s) restored.
Transaction model: UTXO. Balances rebuilt from confirmed rewards and the signed transfer log.
Library Lending Chain (Formative 2) - UTXO model, difficulty 2. Type 'help' for commands.
> pending
Pending pool: 2 unconfirmed record(s) waiting to be mined
# Action Book Title Member Time Reward tx_id
1 BORROWED BK001 Things Fall Apart ALU001 2026-10-04 14:49:25 0 none
2 RETURNED BK001 Things Fall Apart ALU001 2026-10-04 14:49:25 10 c02ef9038572689b...
> mine cloud 2
=== Cloud mining: renting 2 round(s), difficulty 2 ===
Rented hash power : 2000 attempts per round
Rental fee : 20.00 coins per round
Maintenance fee : 10% of every reward earned
Pending records : 2

-- Round 1 --
Block 1: valid hash found (nonce 141)
Confirmed:
[1] BORROWED "Things Fall Apart" (BK001) - John Doe (ALU001)
time=2026-10-04 14:49:25 token_reward=0 tx_id=none (no reward transaction)
hash=007dc07d793fe4bae7a33804c8a455d1db706e19ab009272d260cdd1ee50a67e
prev=005bcde98538ddbba90580bb2626f45456dc4b675bf6e8176a1b305d8bfc5cc01
difficulty=2 nonce=141 signature=VALID
UTXO set (unspent outputs):
ID Owner Amount Created by (txid:vout)
(empty)
Balances: ALU001=0, ALU002=0, ALU003=0
Block 2: valid hash found (nonce 37)
Confirmed:
[2] RETURNED "Things Fall Apart" (BK001) - John Doe (ALU001)
time=2026-10-04 14:49:25 token_reward=10 tx_id=c02ef9038572689b620fb8c849ed304a8dd42d1486ea41263cc1ffc3365bffe9
hash=00c1633ad48d7a76ac849257cb95567555c12748637237819a1fc01246b34717
prev=007dc07d793fe4bae7a33804c8a455d1db706e19ab009272d260cdd1ee50a67e
difficulty=2 nonce=37 signature=VALID
-> reward transaction confirmed: ALU001 credited 10 - fee 1 = 9 coins
UTXO set (unspent outputs):
ID Owner Amount Created by (txid:vout)
#1 ALU001 9 c02ef9038572689b...:0
Balances: ALU001=9, ALU002=0, ALU003=0
Round 1: 180 attempts used, 2 block(s) confirmed, reward 101.00, rental 20.00, maintenance 10.10
cumulative rewards 101.00, cumulative fees 30.10, running net 70.90

-- Round 2 --
Round 2: 0 attempts used, 0 block(s) confirmed, reward 0.00, rental 20.00, maintenance 0.00
cumulative rewards 101.00, cumulative fees 50.10, running net 50.90

--- Cloud mining summary (2 rounds) ---
Blocks confirmed : 2
Gross earnings : 101.00 coins
Total fees paid : 50.10 coins (rental 40.00 + maintenance 10.10)
Net profit : 50.90 coins
> exit

```

Figure 12. A profitable two-round cloud rental with its earnings summary.

```

~/formativeblk/formative-2

$ ./lending_tracker --model utxo --seed 7
Registries loaded: 3 book(s), 3 member(s).
Loaded chain: 1 block(s), integrity verified.
Transaction model: UTX0. Balances rebuilt from confirmed rewards and the signed transfer log.
Library Lending Chain (Formative 2) - UTX0 model, difficulty 2. Type 'help' for commands.
> difficulty 4
Difficulty set to 4: blocks mined from now on need 4 leading zeros (~65536 attempts expected).
> borrow BK002 ALU002
PENDING: BK002 borrowed by ALU002 - queued (not yet on the chain; 1 record(s) waiting to be mined).
> mine cloud 2
=== Cloud mining: renting 2 round(s), difficulty 4 ===
  Rented hash power : 2000 attempts per round
  Rental fee       : 20.00 coins per round
  Maintenance fee  : 10% of every reward earned
  Pending records  : 1

-- Round 1 --
Round 1: 2000 attempts used, 0 block(s) confirmed, reward 0.00, rental 20.00, maintenance 0.00
         cumulative rewards 0.00, cumulative fees 20.00, running net -20.00
         WARNING: rental is unprofitable - cumulative fees (20.00) exceed cumulative rewards (0.00)

-- Round 2 --
Round 2: 2000 attempts used, 0 block(s) confirmed, reward 0.00, rental 20.00, maintenance 0.00
         cumulative rewards 0.00, cumulative fees 40.00, running net -40.00
         WARNING: rental is unprofitable - cumulative fees (40.00) exceed cumulative rewards (0.00)

Note: the block being mined when the rental ended stays in the pending pool.

--- Cloud mining summary (2 rounds) ---
  Blocks confirmed : 0
  Gross earnings   : 0.00 coins
  Total fees paid  : 40.00 coins (rental 40.00 + maintenance 0.00)
  Net profit       : -40.00 coins
  WARNING: this rental was UNPROFITABLE overall.
> exit

```

Figure 13. An unprofitable cloud rental at difficulty 4: the warning appears in every round and in the summary.

10 Security and Immutability

10.1 How the chain is protected

Mechanism	What it protects
SHA-256 over every field	Any change to a book, member, action, reward, <code>tx_id</code> , difficulty or nonce changes the hash
previous_hash links	Changing one block breaks the link to every block after it
Proof of work	Rewriting a block means redoing its mining and the mining of every later block
<code>tx_id</code> check	A block's reward and its transaction identifier must agree
ECDSA signature (P-256)	Only the holder of the library's private key can produce a valid block
Validation at startup	The chain is re-verified before it is trusted; on failure, lending, mining and transfers are refused
Pending pool re-check	Unsigned pending records are re-checked against the registry, reward rules and <code>tx_id</code> on load
Signed transfer log	Transfers are signed and replayed, so spent outputs and used nonces stay used after a restart, and an edited log is rejected

Refusing writes on a compromised chain matters. If new blocks could be mined on top of a tampered chain, they would hash and sign correctly and quietly bury the tampering.

10.2 Tamper demonstrations

Editing a block. In Figure 14, block 2's reward is changed from 10 to 50 directly in `chain.txt`. On startup the program reports that block 2's stored hash no longer matches its recomputed hash, `validate chain` marks the chain COMPROMISED, and a new borrow is refused.

```
~/formativeblk/formative-2
$ cut -d'|' -f1,7,11 data/chain.txt
0|GENESIS|0
1|BORROWED|0
2|RETURNED|10
3|BORROWED|0
4|RETURNED|10
$ sed -i '3s/|10|/|50|/' data/chain.txt # change block 2's reward from 10 to 50
$ ./lending_tracker --model utxo --seed 7
Registries loaded: 3 book(s), 3 member(s).
WARNING: chain integrity check FAILED at block 2: stored hash does not match recomputed hash (block data was modified)
Lending, mining and transfers are disabled; use 'view records' and 'validate chain' to inspect the damage.
Transaction model: UTX0. Balances rebuilt from confirmed rewards and the signed transfer log.
Library Lending Chain (Formative 2) - UTX0 model, difficulty 2. Type 'help' for commands.
> validate chain
Chain status: 5 block(s), COMPROMISED
  first problem at block index 2: stored hash does not match recomputed hash (block data was modified)
> borrow BK002 ALU002
ERROR: chain integrity check failed at block 2 (stored hash does not match recomputed hash (block data was modified)) -
refusing to change anything until this is resolved.
> exit
```

Figure 14. Changing a confirmed reward is detected at startup, and the program refuses to add records.

A smarter attacker. `tests/forge_chain.py` simulates an attacker who knows the format but not the private key. It raises the reward, recomputes the `tx_id`, re-mines block 2 so its hash meets the difficulty again, and then relinks and re-mines every later block. The forged chain passes every hash, link, proof-of-work and `tx_id` check. In Figure 15 it is caught only by the ECDSA signature. This shows why proof of work at a low difficulty is not enough on its own: re-mining three blocks took only 581 attempts, but forging a signature requires the private key.

```
~/formativeblk/formative-2

$ python3 tests/forge_chain.py data/chain.txt 50
Forging block 2: token_reward 10 -> 50
  re-mined block 2 in 91 attempts -> 00a396ed1c80b108...
  re-mined block 3 in 201 attempts -> 0091bec50d71a314...
  re-mined block 4 in 289 attempts -> 003cd5e1b3626619...
  signatures left unchanged (forging them needs the private key)
$ ./lending_tracker --model utxo --seed 7
Registries loaded: 3 book(s), 3 member(s).
WARNING: chain integrity check FAILED at block 2: ECDSA signature is invalid for this block's hash
Lending, mining and transfers are disabled; use 'view records' and 'validate chain' to inspect the damage.
Transaction model: UTXO. Balances rebuilt from confirmed rewards and the signed transfer log.
Library Lending Chain (Formative 2) - UTXO model, difficulty 2. Type 'help' for commands.
> validate chain
Chain status: 5 block(s), COMPROMISED
  first problem at block index 2: ECDSA signature is invalid for this block's hash
> exit
```

Figure 15. A re-mined forgery passes every hash and proof-of-work check but fails signature verification.

Editing the pending pool. Pending records are not signed until they are mined, so they are re-validated when loaded. In Figure 16, giving a pending borrow a 10-coin reward is detected and the program refuses to start.

```
~/formativeblk/formative-2

$ cat data/pending.txt
1791118165|BK002|Americanah|ALU002|Jane Smith|BORROWED|0|
$ sed -i 's/|BORROWED|0|/|BORROWED|10|/' data/pending.txt # give a borrow a reward
$ ./lending_tracker --model utxo --seed 7
Registries loaded: 3 book(s), 3 member(s).
Loaded chain: 5 block(s), integrity verified.
FATAL: pending pool file 'data/pending.txt' contains a record that fails validation (BORROWED BK002 ALU002 reward 10) - it was modified outside the program.
```

Figure 16. A pending record edited to carry a reward is rejected at startup.

10.3 Key management

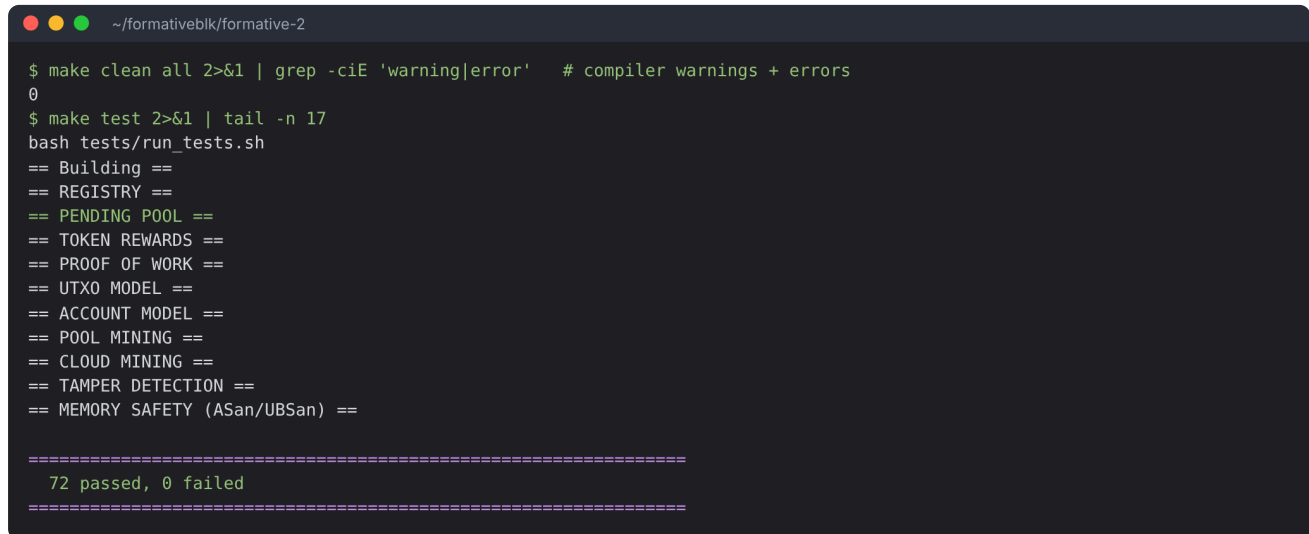
The key pair is created on first run with OpenSSL's `EVP_EC_gen("P-256")`. The private key is written to `keys/private.pem` with owner-only permissions and is excluded from Git. Formative 1 had a weakness: if only `public.pem` was missing, it generated a whole new key pair, overwriting the private key and invalidating every signature on the chain. Formative 2 fixes this by re-deriving the missing public key from the existing private key. A new pair is generated only when there is no private key at all.

10.4 Limitations

- One key signs every block, so the library is a single trusted authority, not a decentralised network. Anyone holding the private key could rewrite and re-sign the chain.
- Difficulty 4 (about 65,536 attempts) is easy for modern hardware. It is set low so the demonstration runs in milliseconds. Real networks use far higher targets.
- Mining is simulated in one process. Pool miners and the cloud rig share the same machine, so hash rates are simulated as attempts per round rather than measured.

11 Testing and Edge-Case Results

`make test` runs 72 automated checks. Every test runs the real compiled program in its own temporary folder with its own data and keys, so tests never touch the real data and do not depend on each other. The suite also rebuilds the program with AddressSanitizer and UndefinedBehaviorSanitizer and runs full sessions in both models to check for memory errors and leaks. The program builds with no warnings under `-Wall -Wextra -Wpedantic`.



```
~/formativeblk/formative-2
$ make clean all 2>&1 | grep -ciE 'warning|error' # compiler warnings + errors
0
$ make test 2>&1 | tail -n 17
bash tests/run_tests.sh
== Building ==
== REGISTRY ==
== PENDING POOL ==
== TOKEN REWARDS ==
== PROOF OF WORK ==
== UTXO MODEL ==
== ACCOUNT MODEL ==
== POOL MINING ==
== CLOUD MINING ==
== TAMPER DETECTION ==
== MEMORY SAFETY (ASan/UBSan) ==

=====
72 passed, 0 failed
=====
```

Figure 17. Zero compiler warnings, and all 72 automated tests passing, including the memory-safety run.

11.1 Edge cases required by the brief

Edge case	Input	Expected	Result	Evidence
Overdue book ("absent student") generates no transaction	borrow BK003 ALU003 , advance 20 , check overdue	OVERDUE record, reward 0, no tx_id ; balance unchanged after mining	Pass	Fig. 4, 11
Insufficient balance (account)	transfer ALU001 ALU002 100 1 with 12 coins	Rejected, balance unchanged	Pass	Fig. 8
Insufficient inputs (UTXO)	transfer ALU001 ALU002 50 with 3 coins	Rejected: inputs $3 < 50 + 1$	Pass	Fig. 7
Failed / reused nonce	Nonce 0 used twice; nonce 3 when 1 is expected	Both rejected, nonce stays at 1	Pass	Fig. 8
Unprofitable cloud rental	difficulty 4 , mine cloud 2	Warning each round and in the summary	Pass	Fig. 13

11.2 Other results

Test	Expected	Result
Unknown book or member ID	ERROR: Book or Member not found , nothing queued	Pass
Borrowing a book that is on loan (confirmed or pending)	Rejected	Pass
Balance before and after mining	0 before, reward – fee after	Pass
Double spend of a UTXO, including after a restart	Rejected	Pass
Change output	Excess of inputs returns to the sender	Pass
Used nonce after a restart	Still rejected (signed transfer log replayed)	Pass
Every block hash meets its difficulty	Leading zeros = stored difficulty	Pass
Difficulty outside 1–4, pool size outside 2–8, rental outside 1–5	Rejected	Pass
Pool shares and attempts add up to the totals	100%, total attempts match	Pass
Unique tx_id for every reward transaction	No duplicates	Pass
Edited block, edited nonce, re-mined forgery, edited pending file, edited transfer log	All detected	Pass
Missing public key	Re-derived; chain still valid	Pass
AddressSanitizer / LeakSanitizer / UBSan	No errors or leaks	Pass

12 Design Choices and Assumptions

- **One lending record per block.** The brief says solo mining "creates a confirmed block", but it also says every Formative 1 block field must stay unchanged, and those fields describe a single lending record. So each mined block holds exactly one record, and one mining run confirms the whole pending pool as consecutive blocks.
- **Two extra block fields for proof of work.** `nonce` and `difficulty` were added alongside the required `token_reward` and `tx_id`. All four are inside the hash.
- **Signing after mining.** Pending records are unsigned. The signature is made over the final proof-of-work hash, which covers every field.
- **A 1-coin fee on every transaction.** This includes rewards, because the brief says the fee is deducted "before crediting the member". Fees go to whoever mines next.
- **Pool rewards include fees.** The brief's formula multiplies by `block_reward`. Here the amount shared is the total of block rewards and fees after the 2% pool fee, so miners are paid for everything the blocks earned.
- **Miners are not members.** Mining rewards are reported but kept out of the member ledgers, which hold only library members.
- **Transfers take effect immediately.** They are not lending records, so they do not wait in the pending pool. They are signed, logged and replayed on startup instead.
- **14-day loan period and a simulated clock.** The brief does not define "late", so a 14-day loan period was chosen. `advance` lets late and overdue cases be shown without waiting.
- **Overdue is a status, not a return.** An OVERDUE book is still on loan and can later be returned for the late reward.
- **Model chosen per session.** The model is chosen with `--model` or the startup menu and applies to every transaction in the session. Balances are rebuilt from the chain, so either model works on the same chain.
- **All numbers in one place.** Rewards, fees, loan period, hash rates and rental costs are all in `src/config.h`.

13 Challenges Encountered and Solutions

13.1 Balances that did not survive a restart

Rewards can always be rebuilt from the chain, but the first version kept member-to-member transfers only in memory. After a restart, an output that had already been spent became unspent again and could be spent a second time, and a used nonce could be used again. The fix was a transfer log in which every transfer is signed with the library's ECDSA key and tagged with the chain length at the moment it happened. On startup, chain rewards and logged transfers are replayed in their original order, so the state is identical every time, and an edited log fails its signature check.

13.2 Duplicate transaction IDs

A transaction ID is derived from the member, book, reward and timestamp. A scripted test that borrowed and returned the same book twice within one second produced two identical reward transactions with the same `tx_id`. Since a `tx_id` must identify one transaction, the later event is now stamped one second later until its ID is unique, and a test checks this.

13.3 Fair pool contribution

The first pool simulation let miners take turns in a fixed order, so the first miner in the list always had the first chance to win and its attempts were always counted in full. The final version runs all miners at the same time on separate nonce slices, picks the miner that found a valid hash earliest in the round, and credits every miner only with the work done up to that moment.

13.4 Testing something random

The test for an unprofitable cloud rental at difficulty 4 failed now and then, because a lucky nonce sometimes finds a block within the rented attempts. The automated test now uses a case that is always unprofitable (renting with an empty pool), and the difficulty 4 case is shown in Figure 13 instead.

13.5 Demonstrating late and overdue books

The on-time, late and overdue rules depend on a 14-day period. The simulated clock solves this without weakening anything: all timestamps come from it, so the stored timestamps and the reward decisions always agree.

13.6 A key-handling bug carried over from Formative 1

While reviewing the Formative 1 code before extending it, I found that deleting only the public key caused a new key pair to be generated, invalidating the whole chain. This was fixed before any Formative 2 work was built on top of it (Section 10.3).

14 Conclusion

The Library Lending Chain now connects lending, token rewards and mining in one flow. Returning a book creates a reward transaction with a SHA-256 identifier, the record waits in a pending pool, and only proof-of-work mining confirms it and credits the member. Both ledger models are implemented with fees, change outputs, double-spend protection, nonces and per-member histories. Solo, pool and cloud mining all confirm the same pending pool and report their rewards, the pool's contribution-based shares, and the cloud rental's profit or loss.

Measuring 240 blocks confirmed that each level of difficulty multiplies the mining effort by about 16. The tamper tests showed that SHA-256 and proof of work make tampering detectable, while the ECDSA signature is what makes it infeasible without the private key. All 72 automated tests pass, including a clean run under AddressSanitizer.

Appendix A: Building and Running

```
sudo apt install build-essential libssl-dev      # OpenSSL 3 development headers
git clone https://github.com/Aman-Kasa/formativeblk.git
cd formativeblk/formative-2
make                                             # builds ./lending_tracker
./lending_tracker --model utxo --difficulty 2    # or --model account; difficulty 1-4
make test                                       # 72 automated tests
```

Command	What it does
borrow <book> <member>	Queue a BORROWED record
return <book>	Queue a RETURNED record and its reward transaction
check overdue / advance <days>	Queue OVERDUE records / move the simulated clock
pending	Show the pending pool
mine solo · mine pool [2-8] · mine cloud <1-5>	Mine the pending pool
difficulty [1-4]	Show or set the difficulty
balances · utxos · history <member>	Show balances, the UTXO set, or an account's history
transfer <from> <to> <amount> [#id...]	UTXO transfer
transfer <from> <to> <amount> <nonce>	Account transfer
view records · validate chain	Show confirmed records / verify the chain